

Dokumentasi Teknis

Facebook ADAS Customer Voice — Pipeline V2

Dokumentasi pembuatan kode & cara kerja: pengumpulan data (scraping) → pembersihan → ekstraksi relevansi → visualisasi Streamlit.

Proyek: **facebook-adas-voice** · versi pipeline **v2.0** · tanggal akhir: 22 September 2026

Ringkasan angka akhir detik ini: 6 kendaraan target · **14.544** raw record → **4.556** unik (dedup **68,67%**) → **116** kandidat ADAS → **39** relevan (PDA 25 / ACC 23) → **19** perlu review kendaraan · **116** tes otomatis lulus · **55/55** validasi final lulus.

1. Pendahuluan & Tujuan

Proyek ini adalah riset *customer voice*: mengumpulkan diskusi komunitas Facebook tentang pengalaman nyata pengguna terhadap sistem ADAS — fokus pada **PDA** (Proactive Driving Assist) dan **ACC** (Adaptive Cruise Control) — untuk 6 kendaraan: Jaecoo J5, Toyota Yaris, Toyota Zenix, Toyota Veloz, XPeng G6, dan BYD Sealion 7. Tujuannya **bukan** penghitungan kata kunci, dan **bukan** perbandingan kendaraan; setiap baris harus dapat ditelusuri kembali ke sumbernya (URL/komunitas/post/komentar).

Repositori memuat **dua lapisan kode** yang bekerja bersama:

- **Lapisan A — Scraper & pipeline v1** (paket `src/`): pengumpulan data dari Facebook (Selenium), parsing, penyimpanan, pembersihan, dan klasifikasi berbasis aturan. Produk utamanya adalah artefak mentah di `data/raw/`.
- **Lapisan B — Pipeline reprocessing V2** (paket `scripts/`): membaca artefak yang sudah ada tanpa scraping ulang, lalu membangun dataset master + hasil analisis per kendaraan + dashboard Streamlit.

Seluruh keluaran V2 berada di `data/output/by_vehicle_v2/`; file mentah **tidak pernah diubah**.

2. Arsitektur Umum

```

Facebook (akun sendiri, terotorisasi)
| `python -m src.main --mode scrape --vehicle "Toyota Yaris"`
▼
src/collectors (facebook_session + noise_filter ringan)
| post → comment → reply (hierarki dipertahankan)
▼
data/raw: raw_data_run_*.csv + raw_data_all.jsonl (append-only)
data/scrapper_state.db (SQLite: registry dedup + resume)
|
— Lapisan B: V2 reprocessing (scripts/, baca data/raw SAJA) —————
▼
audit_facebook_datasets.py      → laporan data/audit/*.json,*.csv
build_master_dataset.py         → master/facebook_master_raw.csv      (14.544)
deduplicate_facebook_data.py    → master/facebook_master_deduplicated.csv (4.556)
clean_facebook_data.py          → master/facebook_master_clean.csv      (4.556)
classify_vehicle.py             → kolom vehicle_v2/confidence/reason (+ 19 review)
filter_adas_relevance.py        → 116 kandidat → 39 relevan / 77 uncertain
classify_experience.py           → fitur, subfitur, skenario, bukti, sentimen, dst.
split_by_vehicle.py             → per-kendaraan + combined + unknown_or_review.csv
generate_summary.py             → processing_summary.json + facebook_master_relevant.csv
▼
Streamlit dashboard (streamlit_app.py) — 6 halaman, Plotly, unduhan CSV/XLSX

```

3. Pengumpulan Data (Scraping)

3.1 Prinsip akses yang sah

Scraper dirancang agar **tidak pernah** melewati login wall. Hanya konten yang dapat dilihat oleh akun pemilik yang dikumpulkan; tidak ada teknik stealth/evasion; CAPTCHA dan kredensial login (`--mode login`) selalu diisi manual oleh pengguna. Jika platform mendeteksi sesi tidak terautentikasi, collector melempar `AuthenticationError`, berhenti, dan menyimpan tangkapan layar + DOM ke `logs/debug/` tanpa menutup data yang sudah terkumpul.

- `python -m src.main --mode login` — membuka Chrome terlihat; pengguna login manual sekali; sesi disimpan sebagai profil Chrome di `data/browser_profiles/facebook/` (dipakai ulang otomatis, termasuk mode headless) dan cookies di `data/cookies_facebook.json`.
- Alternatif: isi `FACEBOOK_COOKIES_FILE` di `.env` dengan ekspor cookies dari sesi sah sendiri.

3.2 Proses koleksi (src/collectors/facebook_collector.py)

Collector berjalan *human-paced* (scroll dengan jeda), memuat halaman komunitas, lalu menyelami hirarki **post** → **komentar** → **balasan**. Pengaturan utama ada di `config/settings.yaml`:

Kunci	Nilai	Fungsi
<code>collection.max_posts</code>	1500	Batas jumlah post per komunitas
<code>collection.max_comments_per_post</code>	60	Batas komentar per post
<code>collection.max_replies_per_comment</code>	4	Batas balasan per komentar
<code>collection.history_since</code>	2022-01-01	Post lebih tua diabaikan (heuristik waktu)

<code>collection.scroll_max_attempts</code> / <code>scroll_pause_seconds</code>	500 / 1.5	Kontrol kecepatan dan ketahanan scroll
<code>collection.retry_max</code> / <code>retry_backoff_base_seconds</code>	3 / 2	Retry dengan backoff eksponensial
<code>collection.headless</code>	true	Mode tanpa jendela saat sudah ada profil

3.3 Penyaringan noise ringan saat scrape

Sebelum disimpan, teks lewat `src/collectors/noise_filter.py` yang **konservatif**: membuang iklan/penjualan yang jelas, spam/ajakan kontak, dan teks asing yang sangat jelas. Teks ambigu **tidak** dibuang agar keputusan akhir diserahkan ke tahap pembersihan. Setiap pembuangan tercatat sebagai statistik di log pipeline (`noise_dropped`).

3.4 Penyimpanan (src/storage)

- Per run → `data/raw/raw_data_run_<timestamp>.csv` (17 artefak CSV)
- Seluruh record → `data/raw/raw_data_all.jsonl` (append-only, satu JSON per baris)
- SQLite `data/scrapper_state.db` → registri post/komentar yang sudah diproses, sehingga re-run bersifat **idempotent** dan bisa *resume* (`python -m src.main --resume`).

Struktur baris mentah diatur `src/models.py` dan mencakup `record_id`, `community`, `community_url`, `post_id`, `comment_id`, `parent_id`, `source_type`, `author_pseudonym`, `date`, `text_raw`, `url`, `scraped_at`. Identitas pengguna dianonimkan (`USER_xxxx`).

4. Audit Dataset (scripts/audit_facebook_datasets.py)

Langkah V2 pertama adalah **mencatat kondisi seluruh artefak mentah**: jumlah sumber (17 file: 16 CSV run + 1 JSONL), baris per sumber, kolom yang tersedia, cakupan per-kendaraan, dan konsistensi terhadap file final lama (`data/output/raw_data_final.csv`). Hasil audit ditulis ke `data/audit/`: **facebook_datasets_audit.csv/json**, **master_union_report.json**, **dedup_report.json**, dan **split_report.json**. Audit menyimpulkan bahwa gabungan seluruh sumber = 14.544 baris dan setelah deduplikasi by identitas = 4.556 unik — sama persis dengan jumlah file `raw_data_final.csv` lama namun dibangun ulang dari lapisan mentah (lebih dapat dipercaya).

5. Pembangunan Master & Deduplikasi (scripts/build_master_dataset.py, deduplicate_facebook_data.py)

5.1 Master dataset

`build_master_dataset.py` menggabungkan semua artefak di `data/raw/` (sesuai `config/analysis.yaml` → `master.raw_dirs`) menjadi `master/facebook_master_raw.csv` (14.544 baris, 17 sumber). Semua kolom dinormalisasi ke urutan standar `STD_COLUMNS` di `scripts/v2_common.py`; teks asli tetap di `text_raw` tanpa perubahan.

5.2 Deduplikasi

Kunci utama deduplikasi adalah `record_id`; bila kosong, dipakai **gabungan** `community` | `post_id` | `comment_id` (fallback). Setiap record yang bertahan diberi `dedup_key` untuk jejak audit. Hasil: **9.988 duplikat dibuang** → **4.556 unik (laju dedup 68,67%)**.

```
# config/analysis.yaml
master:
  raw_dirs: ["data/raw"]
  dedup:
    primary_key: "record_id"
    fallback_key: ["community", "post_id", "comment_id"]
  max_raw_rows_warning: 20000
```

Catatan penting: dedup berbasis **identitas post/komentar**, bukan isi teks. Komentar dengan isi yang sama di thread berbeda tetap menjadi baris terpisah — ini dijelaskan di halaman *Data Quality* dashboard.

6. Pembersihan Teks (scripts/clean_facebook_data.py)

Tahap ini memakai pembersih inti dari lapisan A (`src/cleaners/text_cleaner.py`): de-URL, de-html, normalisasi spasi/whitespace, dan fungsi `normalize_lower` (huruf kecil + aksara/digit alfanumerik untuk pencocokan). Output menambah tiga kolom tanpa menyentuh sumber:

- `text_raw` — teks asli (immutable, selalu dipertahankan)
- `text_clean` — teks dibersihkan untuk tampilan/analisis
- `text_normalized` — teks ternormalisasi untuk pencocokan kata kunci

Eksekusi mencatat jumlah baris yang menjadi kosong setelah pembersihan (hasil akhir: **0** baris kosong dari 4.556) dan menulis `master/facebook_master_clean.csv`.

7. Identifikasi Kendaraan (scripts/classify_vehicle.py + v2_common.py)

Klasifikasi kendaraan bersifat **kontekstual**, bukan sekadar menangkap satu kata. `v2_common` membangun pola alias per kendaraan dari `config/vehicles.yaml`:

- **Pola kuat (strong)** — nama kanonik, nama model, kombinasi *brand+model* (mis. "byd sealion 7"), dan alias yang diawali brand atau berisi spasi/> 5 huruf.
- **Pola pendek (short)** — alias singkat (mis. "zenix"); pola ini hanya memberi bukti tingkat *medium*.

Logika `classify_vehicle` (bentuk disederhanakan):

```
mentions = detect_vehicle_mentions(text, alias_patterns)
src       = canonicalise_record_vehicle(record.vehicle) # prior dari scraper (per komunitas)

jika 1 kendaraan disebut -> vehicle = itu; high (pola kuat) / medium (pola pendek)
                           bila prior konflik -> tetap medium (alasan ...but_context)
jika >1 kendaraan       -> bila prior terdaftar -> prior (medium) [konteks komunitas]
                           bila tidak          -> unknown / low (conflicting)
jika tidak ada mention  -> bila prior ada -> prior (medium) [community_context]
                           bila tidak        -> unknown / low [no_signal]
```

Baris yang hasilnya `unknown`, confidence `low`, atau ber-potensi konflik (`vehicle_reason` memuat `but_context|conflicting`) dirutekan ke `unknown_or_review.csv` untuk ditinjau manual (**19 baris**). Sisa 4.537 baris terdistribusi ke 6 kendaraan.

8. Ekstraksi Relevansi ADAS (scripts/filter_adas_relevance.py)

Ini tahap paling penting dan dirancang agar **kata kunci ≠ kesimpulan**. Ada lapisan pencocokan berurutan yang direpresentasikan sebagai `LayerScorer` — sketsa kode:

```
class LayerScorer:
    def __init__(self, keywords):
        self.patterns = {
            "layer_a_feature_names": ... # "ACC", "PDA", "Adaptive Cruise Control"
            "layer_b_functional": ... # "adaptive cruise", "blind spot monitor", ...
            "layer_c_indonesian": ... # "otomatis ngerem", "ngemudii sendiri", ...
            "combined_adas": ... # "ADAS", "fitur keselamatan", ...
        }
    def match(self, text) -> (dict_hits, list_matched):
        ...
    def classify(self, hits, text) -> (relevance, confidence, reason, feature_hint)
```

Situasi	Label relevansi	Contoh
Layer A (nama fitur eksplisit)	relevan (kadang perlu teks > batas)	"ACC-nya enak dipakai di tol"
Layer B / C (fungsional/RI)	relevan bila teks cukup panjang	"mobilnya ngerem sendiri di tanjakan"
Hanya <code>combined_adas</code> (generik)	uncertain	"mobil ini fitur ADAS-nya lengkap"
Tidak ada hit	not_relevant (bukan kandidat)	"harga ban berapa kak"

Setiap catatan relevan/uncertain menyimpan jejak: `keyword_hits`, `relevance_reason`, `relevance_confidence`, dan `feature_hint`. Pola noise yang **sengaja diselepkkan** dari kandidat/relevan:

- **Persetujuan admin pasar daring** — pola `izin ACC mas admin` berlabel `not_relevant` (rase yang biasanya adalah konfirmasi iklan, bukan suara pengemudi).
- **Akronim kota vs fitur** — `BSD` (Blind Spot Detection) berbenturan dengan nama kota Bintaro Serpong Damai & kawasan sekitarnya (bintaro/depok/ciputat/tangerang/serpong) → demosi ke `not_relevant`/uncertain.
- **Frasa kosmetik** — "baret halus", "goresan halus", "PPF/paint protection" → demosi ke `uncertain`.
- **"follow"** dihapus dari daftar fungsional ACC karena kontaminasi media sosial (mis. "follow Instagram", bukan pengaturan jarak).

Hasil akhir: **116 kandidat** → **39 relevan**, **77 uncertain**, **4.440 not_relevant**.

9. Klasifikasi Pengalaman (`scripts/classify_experience.py`)

Untuk seluruh kandidat, pipeline memanggil klasifikator `softule` dari lapisan A (`src/classifiers/feature_classifier.py`, `src/classifiers/experience_classifier.py`) dengan taksonomi `config/taxonomy.yaml`:

- **feature** — dibatasi ke `PDA | ACC | PDA+ACC | ADAS_OTHER | UNKNOWN` (atas DST `analysis.yaml`)
- **subfeature** — per PDA/ACC (mis. `PDA:curve_assistance`, `PDA:braking`, `ACC:smoothness`)
- **driving_scenario** — tol/highway, macet, stop-and-go, cut-in, dll.
- **evidence_type** — `direct_experience`, `opinion`, `question`, `hearsay`, `specification`, `uncertain`
- **sentiment** — `positive`, `neutral`, `negative`, `mixed`
- **smoothness** — `smooth` / `slightly_abrupt` / `abrupt` / `very_abrupt` / `not_stated`
- **expectation_mismatch** — `yes` / `no` / `unclear`

- **feature_confidence** — tinggi/sedang/rendah (jejak alasan)

Kolom analisis ini hanya diisi untuk kandidat; selain itu memakai nilai default (`UNKNOWN` / `not_stated`) supaya distribusi tidak membingungkan. Distribusi fitur saat ini: **PDA 25, ACC 23**, sisanya UNKNOWN (bukan kandidat). Untuk subset relevan (39): subfitur `PDA:curve_assistance` 14, **not_stated** 13, `PDA:braking` 10, `ACC:smoothness` 1, `ACC:braking` 1.

10. Pemisahan per Kendaraan & Ringkasan (`scripts/split_by_vehicle.py`, `generate_summary.py`)

`split_by_vehicle.py` memecah `facebook_master_clean.csv` menurut `vehicle_v2` dan menulis untuk 6 kendaraan file `raw.csv` (tanpa kolom analisis), `clean.csv`, `relevant.csv`, dan `analyzed.csv` (relevan + kolom analisis). Gabungan semua kendaraan ditulis ke `combined/all_vehicles_{clean,relevant,analyzed}.csv`, serta `unknown_or_review.csv` dan `vehicle_summary.csv`.

`generate_summary.py` merangkum semuanya ke `master/processing_summary.json` dan menulis `master/facebook_master_relevant.csv` (kumpulan relevan lintas kendaraan). Komposisi relevan saat ini:

Kendaraan	Raw (unik)	Kandidat	Relevan
BYD Sealion 7	1.609	—	19
Toyota Veloz	601	—	10
Toyota Zenix	834	—	10
Jaecoo J5	779	—	0
Toyota Yaris	675	—	0
XPeng G6	58	—	0
Total unik	4.556	116	39

Perlu diingat: angka ini adalah *snapshot* kondisi data saat ini. Sebagian kendaraan bernilai 0 relevan karena diskusi yang ada ternyata bernuansa iklan/admin ("izin ACC mas admin") atau kosmetik, bukan pengalaman ADAS — hasil yang jujur, bukan artefak algoritma.

11. Visualisasi Streamlit (`streamlit_app.py`)

Dashboard dibangun dengan Streamlit + Plotly dan **hanya membaca file output nyata** (wajib tanpa angka hardcode). Data dimuat dengan `@st.cache_data:` `processing_summary.json`, `all_vehicles_clean.csv`, `all_vehicles_analyzed.csv`, `unknown_or_review.csv`, `vehicle_summary.csv`.

```
streamlit run streamlit_app.py    # buka http://localhost:8501
```

Enam halaman yang bisa dipilih lewat radio horizontal:

Halaman	Isi
Overview	KPI (unik, kandidat, relevan, review, % dedup) + bar per kendaraan + donut fitur + donut tipe bukti + unduh master

Detail Kendaraan	Pilih kendaraan → KPI, donut fitur, bar per komunitas, sentimen suara pelanggan, tabel catatan relevan + unduh
PDA	Mention PDA per kendaraan, tipe bukti, sentimen, tabel + unduh
ACC	Idem, untuk ACC
Customer Voice	Kutipan per baris (kartu senada), metrik evidence type, donut sentimen, unduh CSV/XLSX
Data Quality	Confidence kendaraan, alasan relevansi, pola teks repetitif (duplikasi konten), tabel review <code>unknown_or_review</code>

Sidebar menyediakan **filter**: kendaraan, fitur, sentimen, tipe bukti (semuanya dalam urutan taksonomi tetap), plus pencarian kata kunci bebas di teks. Unduhan mendukung CSV (`utf-8-sig` agar kompatibel Excel) dan XLSX bila `openpyxl` tersedia. Desain sengaja **tanpa ranking** antar kendaraan dan menaruh catatan bahwa jumlah komentar tidak merepresentasikan kualitas ADAS.

12. Pengujian & Validasi

- **Unit/regresi** — `pytest`: 116 tes lulus, mencakup pencocokan kata kunci, identifikasi kendaraan, deduplikasi, pembersihan, klasifikasi fitur, filter noise, dan tes spesifik V2 (`tests/test_v2_*.py`).
- **Validasi final otomatis** — `scripts/validate_pipeline.py`: **55/55 cek lulus** — keberadaan file, jumlah baris master/dedup/clean, keunikan kunci, kelengkapan 6 kendaraan + `unknown`, nilai enum yang sah (fitur/bukti/sentimen/confidence), konsistensi gabungan vs per-kendaraan, dan keterbacaan laporan.

13. Cara Menjalankan Pipeline V2

```
# Uji regresi
.\venv\Scripts\python.exe -m pytest tests/ -q

# Pipeline penuh (semua tahap; log di logs/pipeline_v2_*.log)
.\venv\Scripts\python.exe scripts\run_pipeline.py

# Paksa ulang semua tahap, atau jalankan satu tahap saja
.\venv\Scripts\python.exe scripts\run_pipeline.py --force
.\venv\Scripts\python.exe scripts\run_pipeline.py --stage clean

# Validasi final 55 cek
.\venv\Scripts\python.exe scripts\validate_pipeline.py

# Dashboard
streamlit run streamlit_app.py
```

14. Keterbatasan & Catatan Penutup

- Dedup berbasis identitas post/komentar — teks identik di thread yang berbeda tetap dihitung terpisah.
- Klasifikasi berbasis aturan + taksonomi (bukan model bahasa); tinjauan manual disarankan pada `unknown_or_review.csv`, `relevant.csv`, dan kolom `*_reason`.
- Dataset mewakili diskusi komunitas yang **terjangkau**, bukan populasi pengguna; angka bukan proksi kualitas ADAS.

- Menjalankan ulang `run_pipeline.py` setelah menambah sumber raw akan memperbarui seluruh angka snapshot.